

Agent Squad Chat Storage Benchmark Report

Corrected local `agent_squad_workload.py` runs · Aerospike Agent Squad adapter vs DynamoDB Local · 2026-09-18

216,000
scheduled calls

216,000
successful calls

100 + 500 QPS
offered per function

0
operational failures

Every scheduled call completed successfully on both backends. The benchmark runner now clamps early-completion response samples to zero rather than dropping them. Aerospike kept response p99 below 5.6 ms at 100 QPS and below 4.4 ms at 500 QPS across all scenarios.

Method and environment

Both runs used identical dimensions: 256-byte payloads, 10-message retained histories, four agents per session, a 1,024-key read pool, 256 maximum in-flight operations, one-second warmup, 30 seconds per function, and deterministic seed `20260915` . Each backend/function pair received 3,000 calls at 100 QPS and 15,000 calls at 500 QPS. Response latency includes scheduler queueing; service latency measures execution only.

Backend	Deployment	Adapter path	100 QPS	500 QPS	Failures
Aerospike	Local Docker, Community Edition 8.1.2.5	Atomic server-side append and trim	18,000 / 18,000	90,000 / 90,000	0
DynamoDB	Local Docker, DynamoDB Local 2.6.1	Whole-conversation read-modify-write	18,000 / 18,000	90,000 / 90,000	0

Scope: This is a local, non-publishable integration comparison. DynamoDB Local does not reproduce managed networking, partitioning, durability, adaptive capacity, throttling, consumed capacity, or service-side latency. These results characterize local adapter behavior on this host, not production database capacity.

100 QPS

Service latency (ms)

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Append one	Aerospike	2.621	3.047	3.211	3.604	4.129	3,000
Append one	DynamoDB	3.506	4.063	4.260	4.719	5.964	3,000
Append pair	Aerospike	2.621	3.113	3.277	3.670	4.653	3,000
Append pair	DynamoDB	3.670	4.194	4.391	4.784	5.964	3,000
Contended	Aerospike	1.671	2.195	2.392	2.851	3.441	3,000
Contended	DynamoDB	4.260	4.915	5.046	5.571	7.012	3,000
Fetch	Aerospike	1.081	1.393	1.491	1.638	2.015	3,000
Fetch	DynamoDB	2.392	2.949	3.080	3.441	4.030	3,000
Fetch all	Aerospike	2.032	2.490	2.589	2.818	3.375	3,000
Fetch all	DynamoDB	3.408	4.063	4.194	4.588	6.291	3,000
Mixed	Aerospike	2.130	3.310	3.604	4.260	4.784	3,000
Mixed	DynamoDB	2.851	4.260	4.456	5.046	6.226	3,000

Response latency (ms)

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Append one	Aerospike	3.375	4.096	4.325	4.850	6.029	3,000
Append one	DynamoDB	7.668	8.913	9.175	9.961	11.010	3,000
Append pair	Aerospike	3.375	4.162	4.391	4.915	6.685	3,000
Append pair	DynamoDB	7.930	9.044	9.437	10.224	14.811	3,000
Contended	Aerospike	2.490	3.670	3.899	4.456	5.308	3,000
Contended	DynamoDB	9.044	10.355	10.617	11.534	12.583	3,000
Fetch	Aerospike	2.097	2.916	3.113	3.408	4.129	3,000
Fetch	DynamoDB	5.439	6.750	7.078	7.733	8.520	3,000

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Fetch all	Aerospike	2.916	3.932	4.260	4.850	5.308	3,000
Fetch all	DynamoDB	7.340	8.651	9.044	9.830	11.272	3,000
Mixed	Aerospike	2.818	4.456	4.784	5.571	6.160	3,000
Mixed	DynamoDB	6.750	8.913	9.437	10.355	11.534	3,000

500 QPS

Service latency (ms)

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Append one	Aerospike	1.638	1.999	2.130	2.818	8.258	15,000
Append one	DynamoDB	1.704	2.720	2.884	3.244	4.325	15,000
Append pair	Aerospike	1.720	2.081	2.195	2.982	9.830	15,000
Append pair	DynamoDB	1.720	2.785	2.982	3.375	4.260	15,000
Contended	Aerospike	1.065	1.393	1.507	1.917	5.046	15,000
Contended	DynamoDB	2.195	3.506	3.768	4.260	5.505	15,000
Fetch	Aerospike	0.606	0.770	0.836	1.040	1.638	15,000
Fetch	DynamoDB	1.409	1.753	1.901	2.228	2.916	15,000
Fetch all	Aerospike	1.180	1.655	1.819	2.228	31.195	15,000
Fetch all	DynamoDB	1.720	2.621	2.851	3.211	5.177	15,000
Mixed	Aerospike	1.573	2.261	2.425	2.916	4.260	15,000
Mixed	DynamoDB	1.671	2.753	2.949	3.277	4.194	15,000

Response latency (ms)

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Append one	Aerospike	1.737	2.392	2.589	3.342	20.185	15,000
Append one	DynamoDB	67.109	169.869	188.744	211.812	224.395	15,000
Append pair	Aerospike	1.982	2.949	3.178	3.899	24.379	15,000
Append pair	DynamoDB	85.983	205.521	232.784	285.213	301.990	15,000

Scenario	Backend	p50	p90	p95	p99	p99.9	Calls
Contended	Aerospike	1.311	2.130	2.425	3.146	20.447	15,000
Contended	DynamoDB	3,355.443	6,106.907	6,308.233	6,710.886	6,710.886	15,000
Fetch	Aerospike	0.967	1.524	1.688	2.392	3.015	15,000
Fetch	DynamoDB	4.719	8.126	9.175	13.369	24.904	15,000
Fetch all	Aerospike	1.475	2.687	3.178	3.867	31.719	15,000
Fetch all	DynamoDB	84.935	171.966	197.132	228.590	247.464	15,000
Mixed	Aerospike	1.737	2.982	3.441	4.325	13.369	15,000
Mixed	DynamoDB	24.379	92.275	108.003	126.878	146.801	15,000

Interpretation

- Both backends completed all 216,000 scheduled calls without operational failures.
- At 100 QPS, Aerospike had lower service and response latency at every reported percentile in all scenarios.
- At 500 QPS, Aerospike had lower response latency at every reported percentile in all scenarios.
- DynamoDB Local completed all scheduled calls but accumulated substantial response queueing at 500 QPS, especially on the contended path; low execution-only service latency does not imply low end-to-end latency.
- The two adapters use different persistence paths, and the local DynamoDB emulator is not representative of managed DynamoDB.

Sources: `benchmarks/agent_squad_workload_2026-09-18_100qps.json` and `benchmarks/agent_squad_workload_2026-09-18_500qps.json`. Agent Squad 1.1.3, agent-squad-aerospike 0.1.0, ai-ecosystem-benchmark 0.0.6. Values are milliseconds reported by the corrected benchmark runner.